



Hello! How is it going?



trynna be best

goo goo ga ga

artur shakirov

notbad

agressive

good

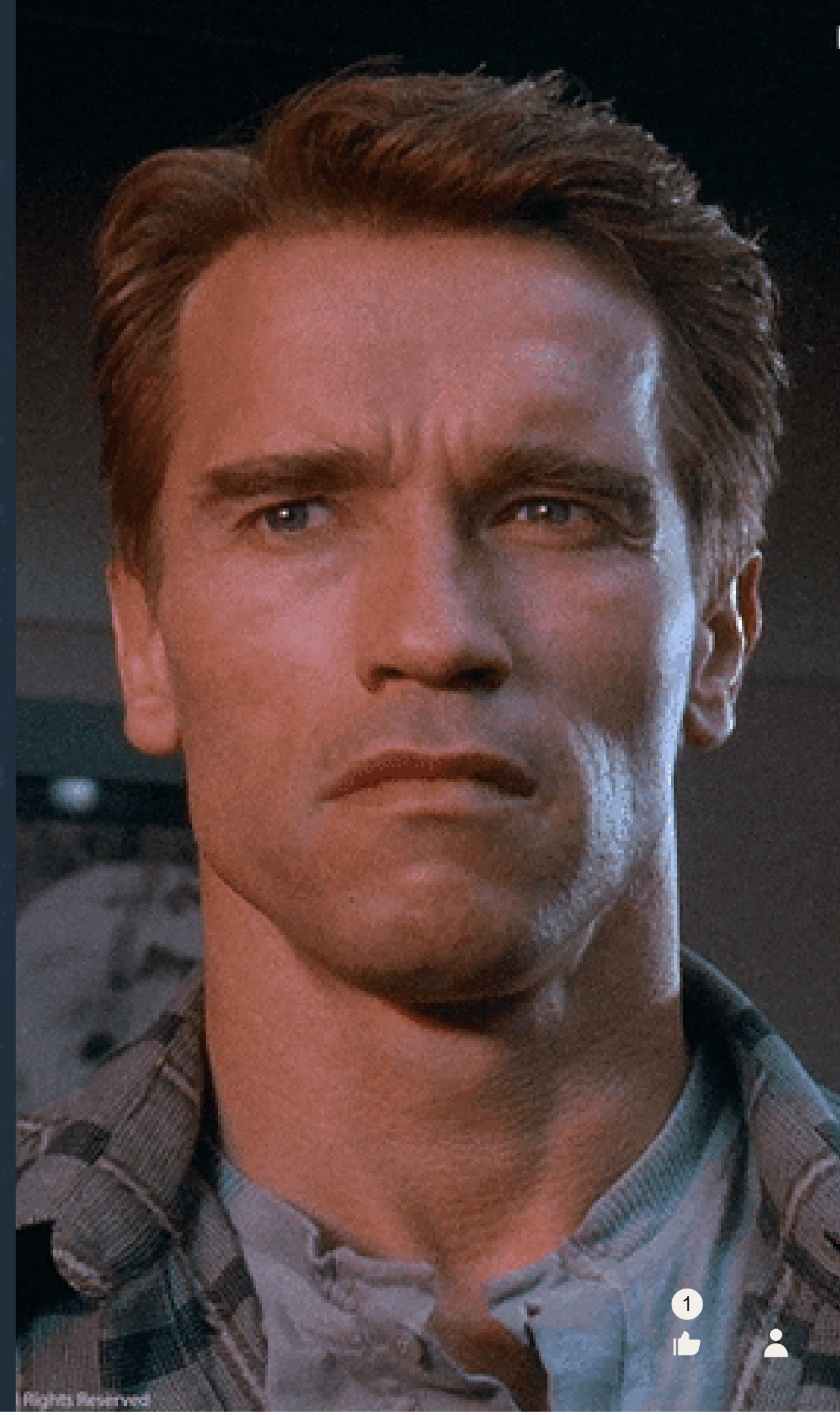
exhilarated fine

sunny

Agenda

- Recall last session
- Discuss current topic
- Practice
- Quiz

Total Recall



What do you remember most from few previous workshops? (short answer)

threads

divide and conquer

Fork Join Service or...

command and conquere

What method recursively splits tasks in Fork/Join?

1 ✗

join()

0 ✗

runTasks()

1 ✗

parallelSplit()

5 ✓

fork()

Which class is the base for a Fork/Join task?

0 ✗

ThreadExecutor

1 ✗

RecursiveGenerationTask

0 ✗

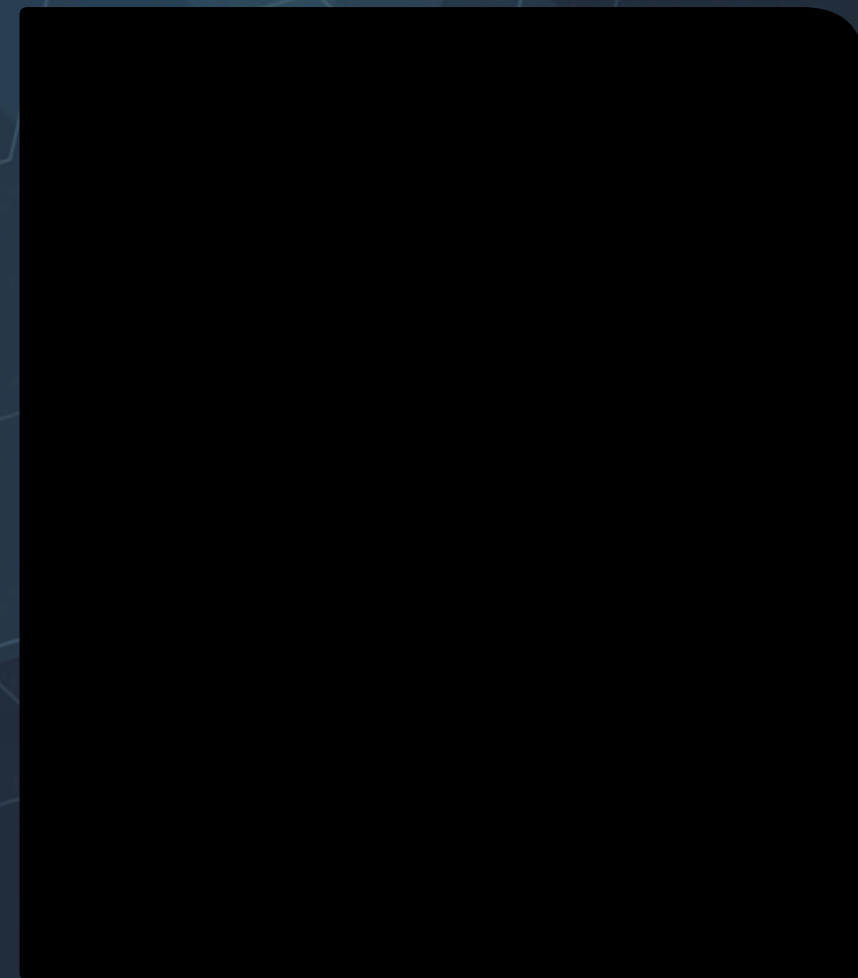
ParallelManager

7 ✓

RecursiveTask or RecursiveAction

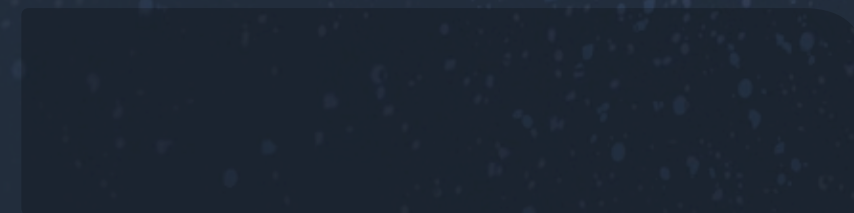
What type of parallelism do Java Streams use?

5 ✓



Implicit parallelism

1 ✗



Explicit concurrency

0 ✗

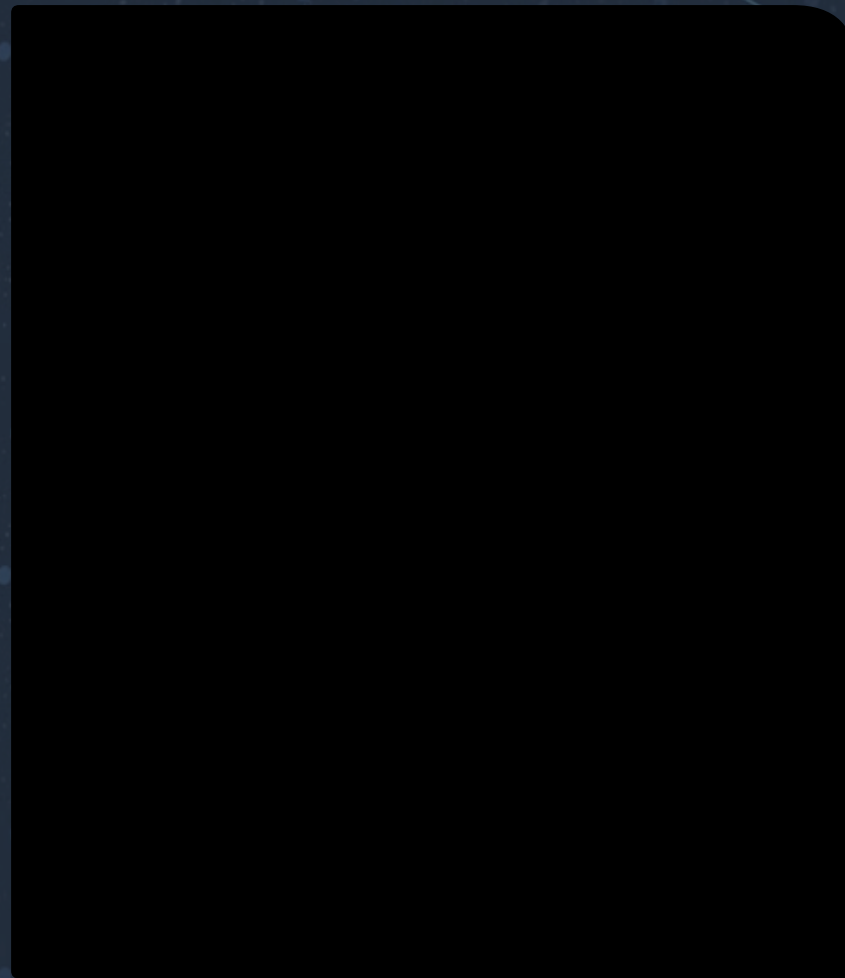
Asynchronous blocking

0 ✗

Thread lifecycle

What does stream.parallel() do?

7 ✓



0 ✗

Starts a thread per operation

0 ✗

Combines blocking threads

0 ✗

Executes atomic locks

Uses Fork/Join for parallel tasks

Next topic

THE P



Random fact

Venus is the hottest planet despite not being closest to the Sun, while Jupiter is large enough to fit all other planets inside



1. Working With JSON

JSON Basics

JSON (JavaScript Object Notation) is a lightweight **data interchange format** used for transmitting data between a server and a web application.

- **Syntax:** Uses key-value pairs (e.g., "key": "value").
- **Primitives:** Includes Strings, Numbers, Booleans, Arrays, and Null.
- **Writing JSON:** You can manually create a JSON string in Java: [Ex1](#)

Simple JSON Implementation

To move beyond manual strings, use libraries like **Jackson** or **Gson**: **Ex2**

- **Add Dependencies (Maven)**
- **Serialize and Deserialize**

JSON Arrays and Collections

Working with lists of objects requires specifying the collection type.

Read JSON Arrays

Ex3

Additional Formatting and Customization

- **Nested JSON:** Simply include objects as fields within other classes; both libraries handle recursion automatically.
- **Configure Names:** Use annotations if JSON keys don't match Java field names.
 - **Gson:** `@SerializedName("note_title")`
 - **Jackson:** `@JsonProperty("note_title")`
- This example demonstrates how to configure both **Jackson** and **Gson** for nested objects, custom field names, and ISO 8601 date formatting:
 - **Jackson:** `mapper.registerModule(new JavaTimeModule());` and disable `WRITE_DATES_AS_TIMESTAMPS`.
 - **Gson:** `new GsonBuilder().setDateFormat("yyyy-MM-dd'T'HH:mm:ssZ").create();`

Ex4

Working with APIs

To fetch JSON from an API, use Java's *HttpClient* to make an **HTTP request** and then parse the response body.

Ex5

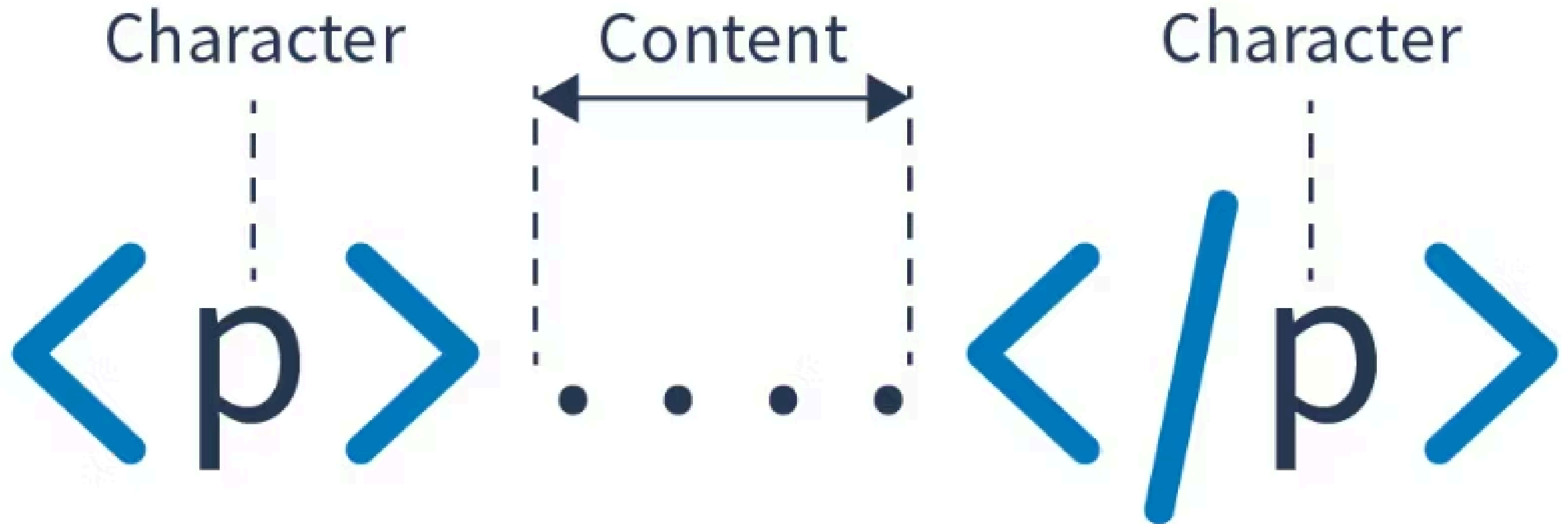
The JSON Parser API, sub_parts :

1. Jackson JSON Java Parser Example
2. Jackson JSON - Converting JSON to Map Example
3. Jackson JSON - Read Specific JSON Key Example
4. Jackson JSON - Edit JSON Document Example
5. Jackson JSON Streaming API Example

from tutorial:

<https://www.digitalocean.com/community/tutorials/jackson-json-java-parser-api-example-tutorial>

2. XML Overview and Schemas



XML

(Extensible Markup Language) is a file format for representing, storing, and transferring arbitrary data. It defines a set of rules for encoding documents in a format that is both human- and machine-readable. It is a text data format with strong support via Unicode for various human languages and is widely used to represent arbitrary data structures.

XML is the foundation of various web technologies such as XHTML, RSS, ATOM, AJAX, and web services, enabling structured data exchange between disparate systems

Advantages:

1. Separates content from presentation
2. Open format with wide application support
3. Works on both the client and server sides
4. Enables data exchange between disparate systems
5. Widespread support in multiple languages/platforms

Disadvantages:

1. Not suitable for very large datasets
2. May not be the most efficient for use in large stores
3. Other formats like JSON may be more efficient in some cases
4. Representing non-textual data can be challenging
5. Complex XML structures can become hard to read

Structure:

XML Document Declaration

The opening declaration should identify a file as an XML document and include attributes such as version, encoding, and standalone.

XML Elements

Also known as tags, they must have valid names that follow specific rules, including not starting with a number and avoiding reserved names like "xml."

Attributes

Specified within opening element tags, they follow similar rules as tag names and appear only once on a given element.

Comments

Used to embed information that is readable by humans, comments are defined with special syntax and can be placed anywhere except inside elem

Character Data Sections (CDATA)

CDATA sections contain unescaped text data that the XML parser skips without parsing. They are useful for including characters that are not legal within parsed XML.

Processing Instructions

Special instructions for the XML parser that are formatted similarly to the document declaration allow XML content to provide instructions that affect parsing.

Entities

Entities help shorten and modularize XML documents by providing markup for characters that would otherwise be illegal. General entities are replaced by the parser with a full string, while character entities represent specific characters using codes or special names.

Rules:

Single Root Tag

XML documents must have a single root tag, which serves as the highest-level element.

Well-Formed

XML documents must adhere to certain syntax rules to be considered well-formed.

Empty Tags

Empty tags must be closed with a slash before the closing angle bracket.

Attribute Values

Attribute values cannot be minimized. XML requires attribute values to be defined explicitly. The attribute values should be enclosed in quotes (single or double).

Proper Nesting

Tags must be properly nested within each other. XML does not allow tags to be partially or incorrectly nested.

"Valid" XML Documents

By employing validation techniques such as DTDs or XML schemas, developers can ensure that XML documents not only contain proper syntax but also follow specific rules, enhancing the reliability and accuracy of XML data.

Document type definitions (DTDs)

- Simple to use, but not very powerful
- Written using a syntax that is different than XML

XML schema

- Much more powerful and flexible than DTD
- Written using XML syntax rules

NameSpaces

XML namespaces provide a mechanism for combining data from different sources or tag sets within an XML document without the risk of naming collisions. They allow for modularity and flexibility when working with extensible XML structures.

In conclusion,

XML provides a powerful and flexible means of structuring and exchanging data. Its syntax, validation mechanisms, namespaces, and transformation capabilities make it suitable for a wide range of applications, including web development, data integration, and more.

Introduction to XML Validation

- **Purpose:** To enforce rules, ensure data integrity, and minimize errors in XML documents.
- **Two Primary Methods:**
 - **DTD (Document Type Definition):** An older, simpler way to define allowed tags, attributes, and content types.
 - **XSD (XML Schema Definition):** A more modern, robust, and complex alternative for enforcing constraints.
- **Key Advantage:** By separating rules (Schema) from data (XML), you reduce the risk of oversight and structural violations.

Anatomy of an XML Schema (XSD)

- **Robustness:** Unlike DTDs, schemas offer a much higher level of control over data types and structures.
- **Separation of Concerns:** Schema files are stored independently from the XML files they validate.
- **Complexity Scale:** Structure can range from basic declarations to highly sophisticated constraints depending on the required level of data control.

Declaring Elements and Attributes

- **Element Declarations:** The core building blocks of a schema that define what tags can exist and what data they can hold.
- **Attribute Declarations:** Used to define metadata for elements.
- **Rule of Placement:** Attributes must be declared **after** any sequence or choice, or at the end of all definitions within a type declaration.

Best Practices and Validation

- **Workflow:**
 1. Define the structure (XSD).
 2. Create the content (XML).
 3. Validate the XML against the XSD.
- **Requirement:** Before validation, the XML content must be "well-formed" (syntactically correct) and error-free.
- **Outcome:** Using XSD results in a flexible, powerful mechanism for maintaining high-quality business data (e.g., electronic business cards).



Quiz time

QUIZ

What does JSON stand for?

3 ✖

Java Serialized Object Notation

0 ✖

Java Syntax Object Nest

10 ✔

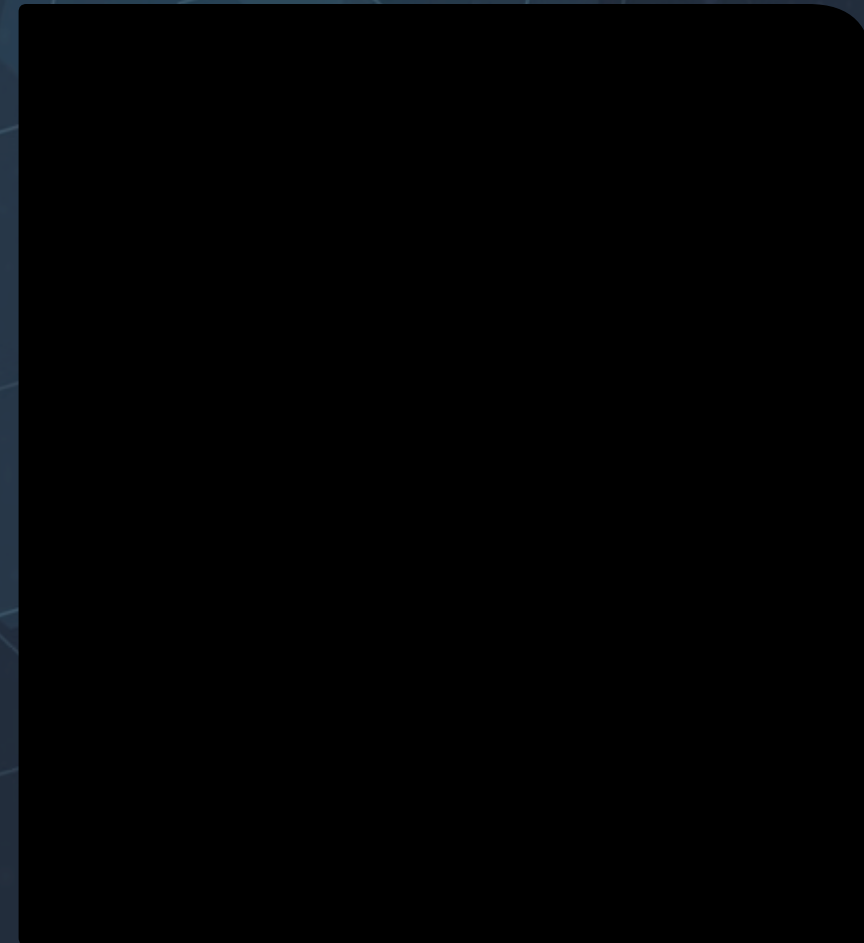
JavaScript Object Notation

0 ✖

Java Secure Object Network

Which Java library handles JSON parsing?

15 



Jackson or Gson

0 

XMLReader

0 

JSONHandler

0 

StringReader

What does a JSON object consist of?

1 ✕

Tags and attributes

0 ✕

Getters and setters

16 ✓

Key-value pairs

0 ✕

Classes and methods

Which format is XML stored in?

0 ✗

Binary

1 ✗

Tabular

1 ✗

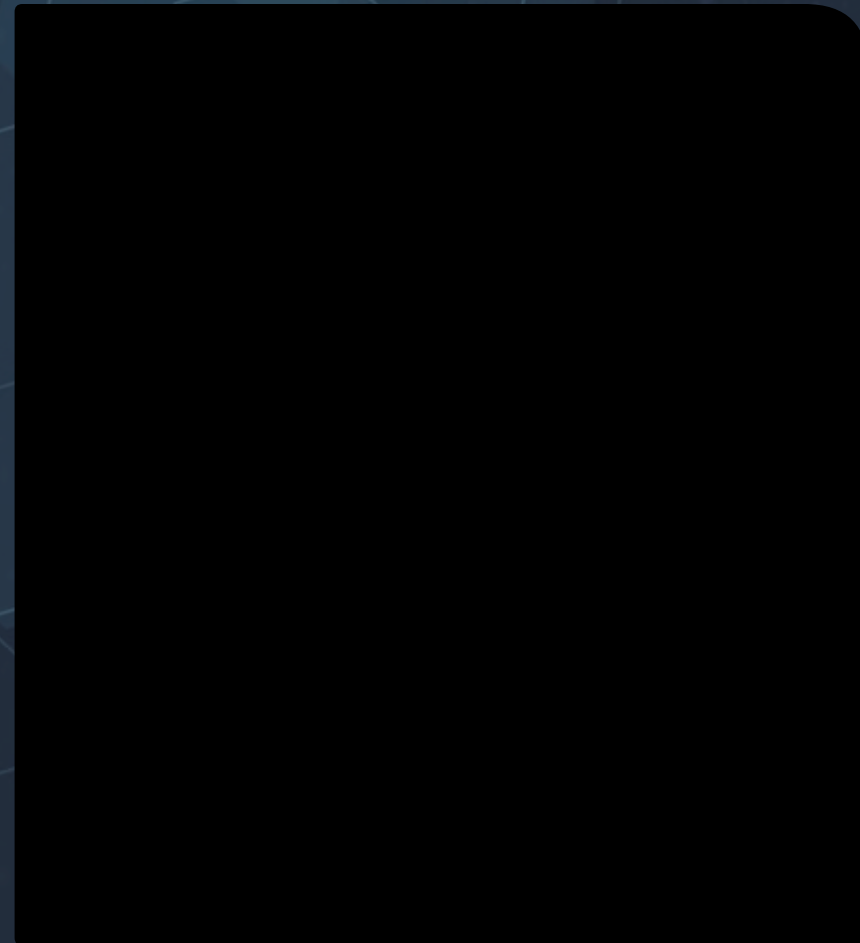
Key-value pairs

15 ✓

Text and tree structure

How is a schema defined in XML?

16 



Using XSD or DTD files

1 

Using JSON attributes

0 

Setting an object parser

0 

Defining key-value relations

What does an XML schema do?

0 ✕

Runs XML programs

1 ✕

Serializes the DOM tree

16 ✓

Defines rules for XML structure

0 ✕

Parses JSON objects

How do you validate XML against a schema?

1 ✗

Using SchemaReader

12 ✓

Use DOM or SAX parser

2 ✗

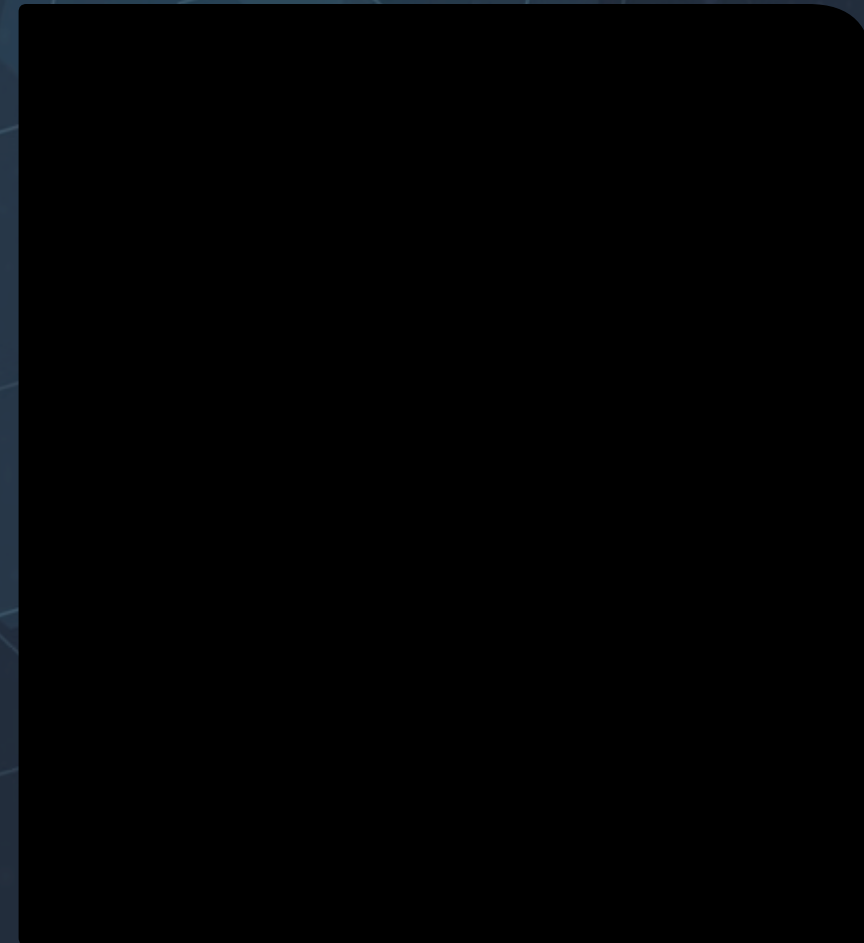
Create new schema objects

2 ✗

Call validateSchema()

Which JSON type represents an ordered list?

15 



Array

1 

Object

1 

Key-value pair

1 

Tree structure

What is true about XML?

0 

Supports only nested objects

0 

Used only with JSON

0 

Requires binary encoding

18 

Self-descriptive and uses tags

Example: What is the key difference between DOM and SAX?

18 ✓



0 ✗

SAX parses concurrently

0 ✗

DOM only reads XML

0 ✗

SAX uses attributes as data keys

DOM loads XML entirely, SAX
reads sequentially

Quiz leaderboard



Q&A part

1 question
0 upvotes

